

An Introduction to Autoencoders

Umberto Michelucci



umberto.michelucci@toelt.ai

January 12, 2022

In this article, we will look at autoencoders. This article covers the mathematics and the fundamental concepts of autoencoders. We will discuss what they are, what the limitations are, the typical use cases, and we will look at some examples. We will start with a general introduction to autoencoders, and we will discuss the role of the activation function in the output layer and the loss function. We will then discuss what the reconstruction error is. Finally, we will look at typical applications as dimensionality reduction, classification, denoising, and anomaly detection.

1 Introduction

Neural networks are typically used in a supervised setting. Meaning that for each training observation $\mathbf{x}_i$ we will have one label or expected value $\mathbf{y}_i$. During training, the neural network model will learn the relationship between the input data and the expected labels. Now suppose we have only unlabeled observations, meaning we only have our training dataset S_T , made of the M observations $\mathbf{x}_i$ with $i = 1, \dots, M$

$$S_T = \{\mathbf{x}_i | i = 1, \dots, M\} \quad (1)$$

Where in general $\mathbf{x}_i \in \mathbb{R}^n$ with $n \in \mathbb{N}$. Autoencoders were first introduced¹ by Rumelhart, Hinton, and Williams in 1986 with the goal of learning

¹You can check Rumelhart, D.E., Hinton, G.E. Williams, R.J.: Parallel distributed processing: Explorations in the microstructure of cognition, Vol. 1. Chap. Learning Internal Representations by Error Propagation, pp. 318-162, MIT Press, Cambridge, MA, USA (1986).

to reconstruct the input observations $\mathbf{x}_i$ with the lowest error possible².

Why would one want to learn to reconstruct the input observations? If you have problems imagining what that means, think of having a dataset made of images. An autoencoder would be an algorithm that can give as output an image that is as similar as possible to the input one. You may be confused, as there is no apparent reason of doing so. To better understand why autoencoders are useful we need a more informative (although not yet unambiguous) definition.

Definition 1 *An autoencoder is a type of algorithm with the primary purpose of learning an "informative" representation of the data that can be used for different applications^a by learning to reconstruct a set of input observations well enough.*

^aBank, D., Koenigstein, N., and Giryas, R., Autoencoders, <https://arxiv.org/abs/2003.05991>

To better understand autoencoders we need to refer to their typical architecture, visualized in Figure 1. The autoencoders' main components are three: an encoder, a latent feature representation, and a decoder. The encoder and decoder are simply functions, while with the name *latent feature representation*, one typically intends a tensor of real numbers (more on that later). Generally speaking, we want the autoencoder to reconstruct the input well enough. Still, at the same time, it should create a latent representation (the output of the **encoder** part in Figure 1) that is useful and meaningful. For example, latent features on hand-written digits³ could be the number of lines required to write each number or the angle of each line and how they connect. Learning how to write numbers certainly does not require to learn the gray values of each pixel in the input image. We humans do not certainly learn to write by filling pixels with gray values. While learning, we extract the essential information that will allow us to solve a problem (writing digits, for example). This latent representation (*how* to write each number) can then be very useful for various tasks (for instance feature extraction that can be then used for classification or clustering) or simply understanding the essential features of a dataset.

²In this article we will discuss at length what we mean with error here.

³Consider for example the MNIST dataset: <http://yann.lecun.com/exdb/mnist/>

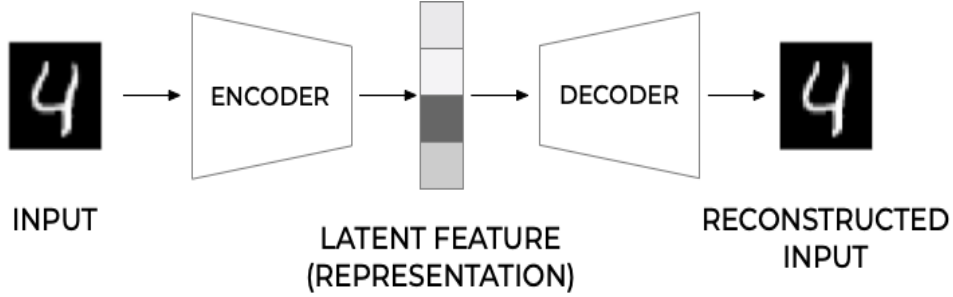


Figure 1: General structure of an autoencoder.

In most typical architectures, the encoder and the decoder are neural networks⁴ (that is the case we will discuss at length in this article) since they can be easily trained with existing software libraries such as TensorFlow or PyTorch with backpropagation. In general, the encoder can be written as a function g that will depend on some parameters

$$\mathbf{h}_i = g(\mathbf{x}_i) \quad (2)$$

Where $\mathbf{h}_i \in \mathbb{R}^q$ (the latent feature representation) is the output of the **encoder** block in Figure 1 when we evaluate it on the input $\mathbf{x}_i$. Note that we will have $g : \mathbb{R}^n \rightarrow \mathbb{R}^q$. The **decoder** (and the output of the network that we will indicate with $\tilde{\mathbf{x}}_i$) can be written then as a second generic function f of the latent features

$$\tilde{\mathbf{x}}_i = f(\mathbf{h}_i) = f(g(\mathbf{x}_i)). \quad (3)$$

Where $\tilde{\mathbf{x}}_i \in \mathbb{R}^n$. Training an autoencoder simply means finding the functions $g(\cdot)$ and $f(\cdot)$ that satisfy

$$\arg \min_{f,g} < [\Delta(\mathbf{x}_i, f(g(\mathbf{x}_i)))] > \quad (4)$$

where Δ indicates a measure of how the input and the output of the autoencoder differ (basically our loss function will penalize the difference between input and output) and $< \cdot >$ indicates the average over all observations. Depending on how one designs the autoencoder, it may be possible

⁴For example, if the encoder and the decoder are linear functions, you get what is called a linear autoencoder. See for more information Baldi, P., Hornik, K.: Neural networks and principal component analysis: Learning from examples without local minima, Neural Netw. 2(1), 53-58 (1989).

to find f and g so that the autoencoder learns to reconstruct the output perfectly, thus learning the identity function. This is not very useful, as we discussed at the beginning of the article, and to avoid this possibility, two main strategies can be used: creating a bottleneck and add regularization in some form.

Note We want the autoencoder to reconstruct the input well enough. Still, at the same time, it should create a latent representation (the output of the encoder) that is useful and meaningful.

Adding a "bottleneck," (more on that later) is achieved by making the latent feature's dimensionality lower (often much lower) than the input's. That is the case that we will look in detail in this article. But before looking at this case, let's briefly discuss regularization.

1.1 Regularization in autoencoders

We will not discuss regularization at length in this article, but we should at least mention it. Intuitively it means enforcing sparsity in the latent feature output. The simplest way of achieving this is to add a ℓ_1 or ℓ_2 regularization term to the loss function. That will look like this for the ℓ_2 regularization term:

$$\arg \min_{f,g} (\mathbb{E} [\Delta(\mathbf{x}_i, g(f(\mathbf{x}_i)))] + \lambda \sum_i \theta_i^2) \quad (5)$$

In the formula the θ_i are the parameters in the functions $f(\cdot)$ and $g(\cdot)$ (you can imagine that in the case where the functions are neural networks, the parameters will be the weights). This is typically easy to implement as the derivative with respect to the parameters are easy to calculate. Another trick that is worth mentioning is to tie the weights of the encoder to the weights of the decoder⁵ (in other words make them equal). Those techniques, and a few others that go beyond the scope of this book, have fundamentally the same effect: add sparsity to the latent feature representation.

We turn now to a specific type of autoencoders: those that build f and g with feed-forward networks that use a bottleneck. The reason for this choice is that they are very easy to implement and are very effective.

⁵For an example in Keras you can check the following page: http://adl.toelt.ai/Chapter25/Your_first_autoencoder_with_Keras.html

2 Feed Forward Autoencoders

A Feed-Forward Autoencoder (FFA) is a neural network made of dense layers⁶ with a specific architecture, as can be schematically seen in Figure 2.

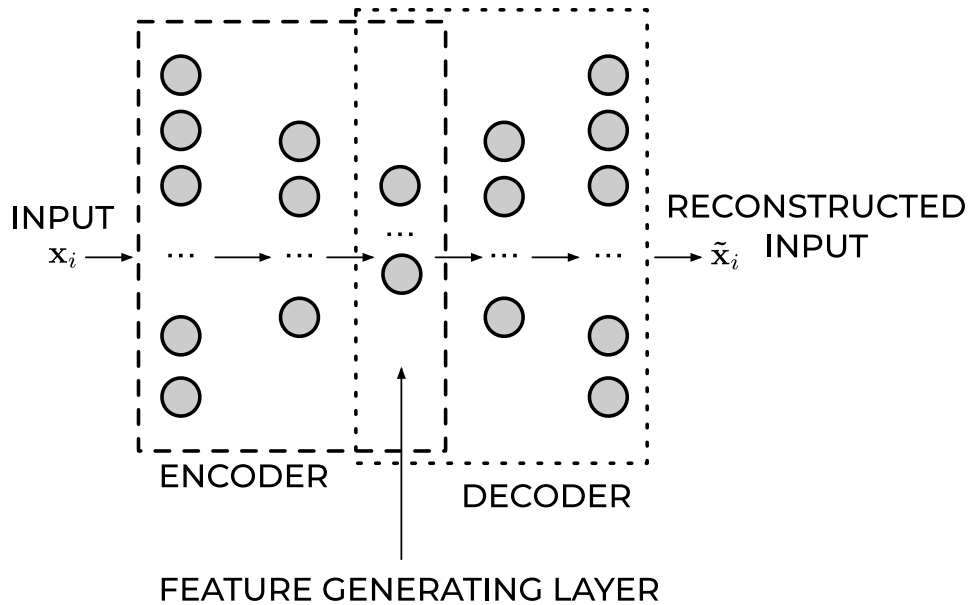


Figure 2: A typical architecture of a Feed-Forward Autoencoder. The number of neurons in the layers at first goes down as we move through the network until it reaches the middle and then starts to grow again until the last layer has the same number of neurons as the input dimensions.

A typical FFA architecture (although it is no mandatory requirement) has an odd number of layers and is symmetrical with respect to the middle layer. Typically, the first layer has a number of neurons $n_1 = n$ (the size of the input observation $\mathbf{x}_i$). As we move toward the center of the network, the number of neurons in each layer drops in some measure. The middle layer (remember we have an odd number of layers) usually has the smallest number of neurons. The fact that the number of neurons in this layer is smaller than the size of the input, is the **bottleneck** we mentioned earlier.

⁶A dense layer is simply a set of neurons that gets their inputs from the previous layer. Each neuron in a dense layer gets as input the output of all neurons in the previous layer.

In almost all practical applications, the layers after the middle one are a mirrored version of the layers before the middle one. For example, an autoencoder with 3 layers could have the following numbers of neurons: $n_1 = 10$, $n_2 = 5$ and then $n_3 = n_1 = 10$ (supposing we are working on a problem where the input dimension is $n = 10$). All the layers up to and including the middle one, make what is called the encoder, and all the layers from and including the middle one (up to the output) make what is called the decoder, as you can see depicted in Figure (25.2). If the FFA training is successful, the result will be a good approximation of the input, in other words $\tilde{\mathbf{x}}_i \approx \mathbf{x}_i$. What is essential to notice is that the decoder can reconstruct the input by using only a much smaller number (q) of features than the input observations initially have (n). The output of the middle layer $\mathbf{h}_i$ are also called a *learned representation* of the input observation $\mathbf{x}_i$.

Note The encoder can reduce the number of dimensions of the input observation (n) and create a learned representation ($\mathbf{h}_i$) of the input that has a smaller dimension $q < n$. This learned representation is enough for the decoder to reconstruct the input accurately (if the autoencoder training was successful as intended).

2.1 Activation Function of the Output Layer

In autoencoders based on neural networks, the output layer's activation function plays a particularly important role. The most used functions are ReLU and sigmoid. Let us look at both and look at some tips on when to use which and why you should choose one instead of the other.

2.1.1 ReLU

The ReLU activation function can assume all values in the range $[0, \infty]$. As a remainder, its formula is

$$\text{ReLU}(x) = \max(0, x). \quad (6)$$

This choice is good when the input observations $\mathbf{x}_i$ assume a wide range of positive values. If the input $\mathbf{x}_i$ can assume negative values, the ReLU is, of course, a terrible choice, and the identity function is a much better choice.

Note ReLU activation function for the output layer is well suited for cases when the input observations $\mathbf{x}_i$ assume a wide range of positive real values.

2.1.2 Sigmoid

The sigmoid function σ can assume all values in the range $]0, 1[$. As a reminder its formula is

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (7)$$

This activation function can only be used if the input observations $\mathbf{x}_i$ are all in the range $]0, 1[$ or if you have normalized them to be in that range. Consider as an example the MNIST dataset. Each value of the input observation $\mathbf{x}_i$ (one image) is the gray values of the pixels that can assume any value from 0 to 255. Normalizing the data by dividing the pixel values by 255 would make each observation (each image) have only pixel values between 0 and 1. In this case, the sigmoid would be a good choice for the output layer's activation function.

Note The sigmoid activation function for the output layer is a good choice in all cases where the input observations assume only values between 0 and 1 or if you have normalized them to assume values in the range $]0, 1[$.

2.2 Loss Function

As with any neural network model, we need a loss function to minimize. This loss functions should measure how big is the difference between the input $\mathbf{x}_i$ and output $\tilde{\mathbf{x}}_i$. If you remember the explanations at the beginning, you will realize that our loss function will be

$$\mathbb{E} [\Delta(\mathbf{x}_i, g(f(\mathbf{x}_i)))] . \quad (8)$$

Where for FFAs, g , and f will be the functions that are obtained with dense layers, as discussed in the previous sections. Remember that an autoencoder is trying to learn an approximation of the identity function; therefore,

you want to find the weights in the network that gives you the smallest difference according to some metric ($\Delta(\cdot)$) between $\mathbf{x}_i$ and $\tilde{\mathbf{x}}_i$. Two loss functions are widely used for autoencoders: Mean Squared Error (MSE) and Binary Cross-Entropy (BCE). Let us have a more in-depth look at both since they can only be used when specific requirements are met.

2.2.1 Mean Square Error

Since an autoencoder is trying to solve a regression problem, the most common choice as a loss function is the Mean Square Error (MSE):

$$L_{\text{MSE}} = \text{MSE} = \frac{1}{M} \sum_{i=1}^M |\mathbf{x}_i - \tilde{\mathbf{x}}_i|^2 \quad (9)$$

The symbol $|\cdot|$ indicates the norm of a vector⁷, and M is the number of the observation in the training dataset. It can be used in almost all cases, independently of how you choose your output layer activation function or how you normalize the input data. It is easy to show that the minimum of L_{MSE} is found for $\tilde{\mathbf{x}}_i = \mathbf{x}_i$. To prove it, let us calculate the derivative of L_{MSE} with respect to a specific observation j . Remember that the minimum is found when the condition

$$\frac{\partial L_{\text{MSE}}}{\partial \tilde{x}_j} = 0 \quad (10)$$

is met for all $i = 1, \dots, M$. To simplify the calculations, let us assume that the inputs are one dimensional⁸ and let us indicate them with x_i . We can write

$$\frac{\partial L_{\text{MSE}}}{\partial \tilde{x}_j} = -\frac{2}{M} (x_j - \tilde{x}_j) \quad (11)$$

Equation 1 is satisfied when $x_j = \tilde{x}_j$ as can be easily seen from Equation (25.1), as we wanted to prove. To be precise, we also need to show that

$$\frac{\partial^2 L_{\text{MSE}}}{\partial \tilde{x}_j^2} > 0 \quad (12)$$

⁷The norm of a vector is simply the square root of the sum of the square of the components.

⁸If we did not make this assumption, one would have to calculate the gradient of the loss function instead of the simple derivative.

This is easily proved as we have

$$\frac{\partial^2 L_{MSE}}{\partial \tilde{x}_j^2} = \frac{2}{M} \quad (13)$$

that is greater than zero, therefore confirming our assumption that for $x_j = \tilde{x}_j$ we indeed have a minimum.

2.2.2 Binary Cross-Entropy

If the activation function of the output layer of the FFA is a sigmoid function, thus limiting neuron outputs to be between 0 and 1, and the input features are normalized to be between 0 and 1 we can use as loss function the binary cross-entropy, indicated here with L_{CE} . Note that this loss function is typically used in classification problems, but it works beautifully for autoencoders. The formula for it is

$$L_{CE} = -\frac{1}{M} \sum_{i=1}^M \sum_{j=1}^n [x_{j,i} \log \tilde{x}_{j,i} + (1 - x_{j,i}) \log(1 - \tilde{x}_{j,i})] \quad (14)$$

Where $x_{j,i}$ is the j^{th} component of the i^{th} observation. The sum is over the entire set of observations and over all components of the vectors. Can we prove that minimizing this loss function is equivalent to reconstructing the input as well as possible? Let us calculate where L_{CE} has a minimum with respect to $\tilde{\mathbf{x}}_i$. In other words, we need to find out what values should $\tilde{\mathbf{x}}_i$ assume to minimize L_{CE} . As we have done for the MSE, to make the calculations easier, let us consider the simplified case where $\mathbf{x}_i$ and $\tilde{\mathbf{x}}_i$ are one-dimensional and let us indicate them with x_i and $\tilde{x}_i$.

To find the minimum of a function, as you should know from calculus, we need the first derivative of L_{CE} . In particular we need to solve the set of M equations

$$\frac{\partial L_{CE}}{\partial \tilde{x}_i} = 0 \quad \text{for } i = 1, \dots, M \quad (15)$$

In this case it is easy to show that the binary cross-entropy L_{CE} is minimized when $x_i = \tilde{x}_i$ for $i = 1, \dots, M$. Note that strictly speaking, this is true only for x_i different than 0 or 1 since $\tilde{x}_i$ can be neither 0 nor 1. To find when the

L_{CE} is minimized we can derive L_{CE} with respect to a specific input $\tilde{x}_j$

$$\begin{aligned}\frac{\partial L_{CE}}{\partial \tilde{x}_j} &= -\frac{1}{M} \left[\frac{x_j}{\tilde{x}_j} - \frac{1-x_j}{1-\tilde{x}_j} \right] = -\frac{1}{M} \left[\frac{x_j(1-\tilde{x}_j) - \tilde{x}_j(1-x_j)}{\tilde{x}_j - \tilde{x}_j^2} \right] = \\ &= -\frac{1}{M} \left[\frac{x_j - \tilde{x}_j}{\tilde{x}_j - \tilde{x}_j^2} \right]\end{aligned}\tag{16}$$

Now remember that we need to satisfy the condition

$$\frac{\partial L_{CE}}{\partial \tilde{x}_j} = 0\tag{17}$$

That can happen only if $x_j = \tilde{x}_j$ as can be seen from Equation (25.2). To make sure that this is a minimum we need to evaluate the second derivative. Since the point for which the first derivative is zero is a minimum only if

$$\frac{\partial^2 L_{CE}}{\partial \tilde{x}_j^2} > 0\tag{18}$$

We can calculate the second derivative at the minimum point $x_j = \tilde{x}_j$ easily

$$\left. \frac{\partial^2 L_{CE}}{\partial \tilde{x}_j^2} \right|_{x_j=\tilde{x}_j} = -\frac{1}{M} \left[\frac{x_j(2\tilde{x}_j-1) - \tilde{x}_j^2}{(1-\tilde{x}_j^2)\tilde{x}_j^2} \right] \bigg|_{x_j=\tilde{x}_j} = \frac{1}{M} \left[\frac{\tilde{x}_j(1-\tilde{x}_j)}{(1-\tilde{x}_j^2)\tilde{x}_j^2} \right]\tag{19}$$

Now remember that $\tilde{x}_j \in]0, 1[$. We can immediately see that the denominator of the previous formula is greater than zero. The nominator is also clearly greater than zero since $1 - \tilde{x}_i > 0$. Dividing two positive numbers gives a positive number, thus we have just proved that

$$\frac{\partial^2 L_{CE}}{\partial \tilde{x}_i^2} > 0\tag{20}$$

The minimum of the cost function is reached when the output is exactly equal to the inputs, as we wanted to prove.

Note an essential prerequisite for using the binary cross-entropy loss function is that the inputs **must** be normalized between 0 and 1 and that the activation function for the last layer must be a *sigmoid* or *softmax* function.

2.3 Reconstruction Error

Note: it's not loss function

The reconstruction error (RE) is a metric that gives you an indication of how good (or bad) the autoencoder was able to reconstruct the input observation $\mathbf{x}_i$. The most typical RE used is the MSE

$$\text{RE} \equiv \text{MSE} = \frac{1}{M} \sum_{i=1}^M |\mathbf{x}_i - \tilde{\mathbf{x}}_i|^2 \quad (21)$$

That can be easily calculated. The RE is used often when doing anomaly detection with autoencoders, as we will explain later. There is an easy intuitive explanation of the reconstruction error. When the RE is significant, the autoencoder could not reconstruct the input well, while when it is small, the reconstruction was successful. Figure 3 shows an example of big and small reconstruction errors when an autoencoder tries to reconstruct an image.

2.4 Example: reconstruction of hand-written digits

Let us now see how an autoencoder performs with a real example, using the MNIST dataset. This dataset⁹ contains 70000 hand-written digits from 0 to 9. Each image is 28×28 pixels with only gray values, that means that we have 784 features (the pixel gray values) as inputs. Let us start with an autoencoder with 3 layers with the numbers of neurons in each layer equal to (784, 16, 784). Note that the first and last layers must have a dimension equal to the input dimensions. For this example, we used the Adam optimizer¹⁰, as loss function the cross-entropy¹¹ and we trained the model for 30 epochs with a batch size of 256. In Figure 4 you can see two lines of images of digits. The line at the top contains ten random images from the original dataset, while the ones at the bottom are the reconstructed images with the autoencoder we just described.

It is impressive that to reconstruct an image with 784 pixels, 10 classes and 70000 images only 16 features are needed to have a result that, although

⁹More information on the dataset can be found here: <http://yann.lecun.com/exdb/mnist/>.

¹⁰You can find the entire code at the address <https://adl.toelt.ai>.

¹¹In this case we normalized the input features to be between 0 and 1.

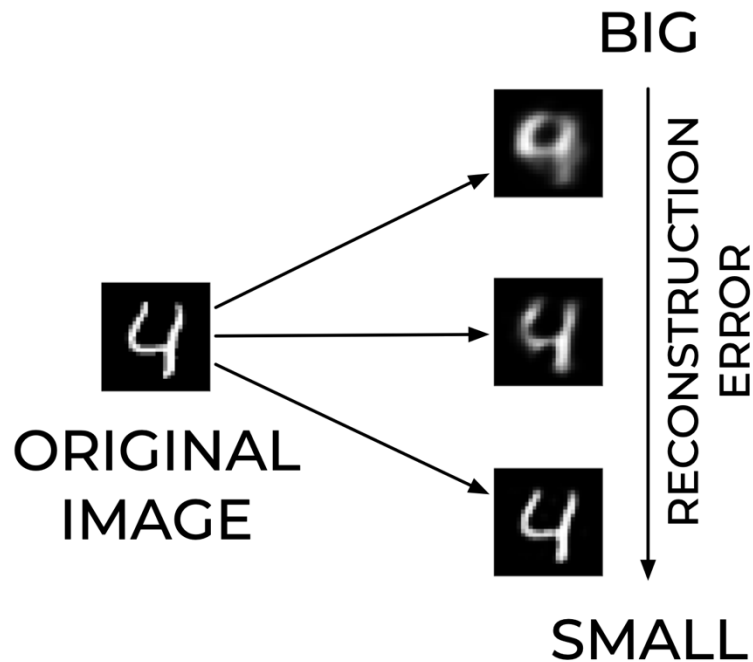


Figure 3: An example of big and small reconstruction error when an autoencoder tries to reconstruct an image.

not perfect, allows us to understand almost entirely what digit was used as input. Increasing the middle layer's size to 64 (and leaving all other parameters the same) gets a much better result as you can see in Figure 5.

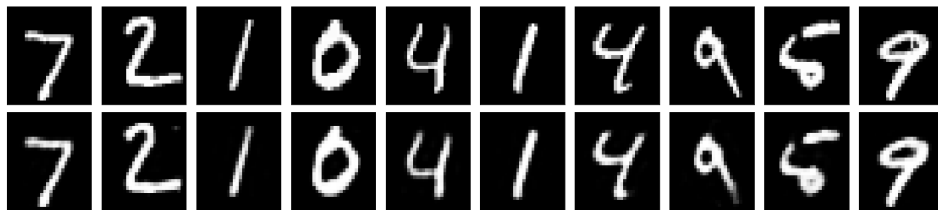


Figure 5: In the top line you can see the original digits from the MNIST dataset. While the line below are the digits reconstructed by the autoencoder with number of neurons equal to (784, 64, 784).

This tells us that the relevant information on how to write digits is con-

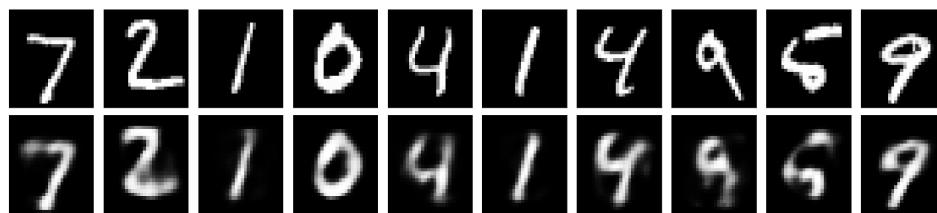


Figure 4: In the top line, you can see the original digits from the MNIST dataset. In contrast, the line below contains the digits reconstructed by the autoencoder with number of neurons equal to (784, 16, 784).

tained in a much lower number of features than 784.

Note An autoencoder with a middle layer smaller than the input dimensions (a bottleneck) can be used to extract the essential features of an input dataset creating a learned representation of the inputs given by the function $g(\mathbf{x}_i)$. Effectively an FFA can be used to perform dimensionality reduction.

The FFA will not recreate the input digits well if the number of neurons in the middle layer is reduced too much (if the bottleneck is too extreme). Figure 6 shows the reconstruction of the same digits with an autoencoder with only 8 neurons in the middle layer. With only 8 neurons in the middle layer, you can see that some reconstructed digits are wrong. As you can see in Figure 6 the 4 is reconstructed as a 9 and a 2 is reconstructed to something that resembles a 3. In Figure 7, you can compare the reconstructed digits by all

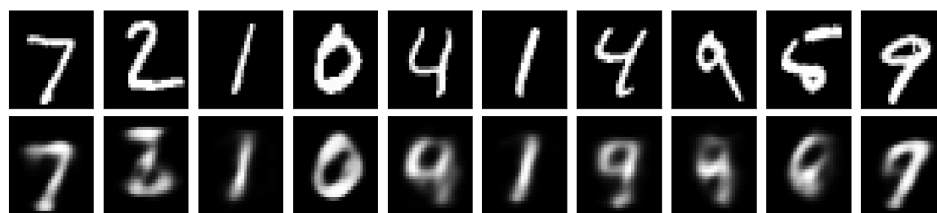


Figure 6: In the top line you can see the original digits from the MNIST dataset. In contrast, the line below contains the digits reconstructed by the autoencoder with number of neurons equal to (784, 8, 784).

the FFAs we have discussed. From Figure 7 you can see how, increasing the

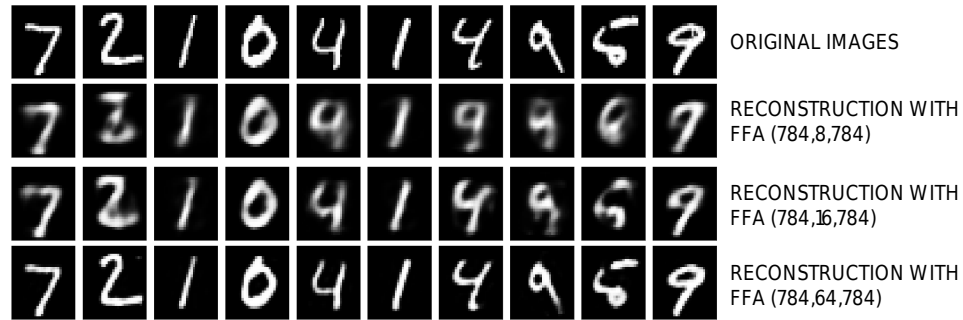


Figure 7: In the top line, you can see the original digits from the MNIST dataset. The second line of digits contains the digits reconstructed by the FFA (784,8,784), the third by the FFA (784,16,784), and the last one by the FFA (784,64,784).

middle layer's size, the reconstruction gets better and better, as we expected.

For these examples, we have used the binary cross entropy as loss function, but the MSE would have worked also well, and results can be seen in Figure 2.4.



Figure 8: In the top line, you can see ten random original digits from the MNIST dataset. The second line of digits contains the digits reconstructed with an FFA with 16 neurons in the middle layer and the binary cross-entropy as the loss function. The last line contains images reconstructed with the MSE as loss function.

3 Autoencoders Applications

3.1 Dimensionality Reduction

As we mentioned in this article, using the bottleneck method, the latent features will have a dimension q that is smaller than the dimensions of the input observations n . The *encoder* part (once trained) does naturally (by design) dimension reduction producing q real numbers. One can use the latent features for various tasks, such as classification (as we will see in the next section) or clustering. We would like to point out some of the advantages of dimensionality reduction with an autoencoder compared to a more classical PCA approach. The autoencoder has one main benefit from a computational point of view: it can deal with a very big amount of data efficiently since its training can be done with mini-batches, while PCA, one of the most used dimensionality reduction algorithms, needs to do its calculations using the entire dataset. PCA is an algorithm that projects a dataset on the eigenvectors of its covariance matrix¹², thus providing a linear transformation of the features. Autoencoders are more flexible and consider non-linear transformations of the features. The default PCA method uses $\mathcal{O}(d^2)$ space for data in $\mathbb{R}^d$. This is, in many cases, not computationally feasible, and the algorithm does not scale up with increasing dataset size. This may seem irrelevant, but in many practical applications, the amount of data and the number of features is so big that PCA is not a practical solution from a computational point of view.

Note The use of an autoencoder for dimensionality reduction has one main advantage from a computational point of view: it can deal with a very big amount of data efficiently since its training can be done with mini-batches.

Equivalence with PCA It is not a very known results, but one worth to mention, that a FFA is equivalent to PCA if the following conditions are met:

¹²Akshay Balsubramani, Sanjoy Dasgupta, and Yoav Freund. The fast convergence of incremental pca. In Advances in neural information processing systems, pages 3174–3182, 2013.

- We use a linear function for the encoder $g(\cdot)$
- We use a linear function for the decoder $f(\cdot)$
- We use the MSE for the loss function
- We normalize the inputs to

$$\hat{x}_{i,j} = \frac{1}{\sqrt{M}} \left(x_{i,j} - \frac{1}{M} \sum_{k=1}^M x_{k,j} \right) \quad (22)$$

The proof is long and can be found in the notes by M.M. Kahpra for the course CS7015 (Indian Institute of Technology Madras) at this link <http://toe.lt/1a>.

3.2 Classification Go to 'AutoEncoders.pdf', page 11

3.2.1 Classification with Latent Features

Let us now suppose that we want to classify our input images of the MNIST dataset. We can simply use all the features, in our case, the 784 pixel values of the images. We can simply use an algorithm as kNN for illustrative purposes. Doing it with 7 nearest neighbors on the training MNIST dataset (with 60000 images) will take around 16.6 minutes¹³ (ca. 1000 sec) and gets you an accuracy on the test dataset of 10000 images of 96.4%¹⁴. However, what happens if we use this algorithm not with the original dataset, but with the latent features $g(\mathbf{x}_i)$? For example, if we consider an FFA with 8 neurons in the middle layer and again train a kNN algorithm on the latent features $g(\mathbf{x}_i) \in \mathbb{R}^8$ we get an accuracy of 89% in 1.1 sec. We get a gain of a factor of 1000 in running time, for a loss of 7.4% in accuracy¹⁵ (see Table 1).

Using 8 features allow us to get a very high accuracy in just one second. We can do the same analysis with another dataset, the Fashion MNIST¹⁶ dataset (a dataset from Zalando very similar to the MNIST one, only with

¹³The examples have been run on Google Colab.

¹⁴Note that for these examples the accuracy is calculated applying the trained model on the test dataset, while the running time is the time needed to train the algorithm on the training dataset.

¹⁵You can run those tests yourself going to article 25 at <https://adl.toelt.ai>.

¹⁶<https://research.zalando.com/welcome/mission/research-projects/fashion-mnist/>

Input Data	Accuracy	Running Time
Original data $\mathbf{x}_i \in \mathbb{R}^{784}$	96.4%	1000 sec. $\approx$ 16.6 min.
Latent Features $g(\mathbf{x}_i) \in \mathbb{R}^8$	89%	1.1 sec.

Table 1: the different in accuracy and running time when applying the kNN algorithm to the original 784 features or the 8 latent features for the MNIST dataset.

clothing images instead of hand-written digits) for illustrative purposes. The dataset has, as the MNIST one, 60000 training images and 10000 test ones. In Table 2 you can see the summary of the results of applying kNN to the testing portion of this dataset. It is exciting to note that with an FFA

Input Data	Accuracy	Running Time
Original data $\mathbf{x}_i \in \mathbb{R}^{784}$	85.4%	1040 sec. $\approx$ 16.6 min.
Latent Features $enc(\mathbf{x}_i) \in \mathbb{R}^8$	79.9%	1.2 sec.
Latent Features $enc(\mathbf{x}_i) \in \mathbb{R}^{16}$	83.6%	3.0 sec.

Table 2: the difference in accuracy and running time when applying the kNN algorithm to the original 784 features with a FFA with 8 neurons and with a FFA with 16 neurons for the Fashion MNIST dataset.

with 16 neurons in the middle layer, we reach an accuracy of 83.6% in just 3 sec. When applying a kNN algorithm to the original features (784), we get an accuracy only 1.8% higher but with a running time ca. 330 times longer.

Note Using autoencoders and doing classification with the latent features is a very viable technique to reduce the training time by several order of magnitude while incurring a minor drop in accuracy.

3.2.2 Curse of dimensionality – a small detour

Is there any other reason why we want to do dimensionality reduction before doing any classification? Reducing running time is one reason, but another important one plays a significant role when the input dimension is very large,

i.e., the datasets that have a very high number of features: the curse of dimensionality. To understand why we need to make a quick detour in the problem of high dimensionality classification and discuss the *curse of dimensionality*. Let us consider the unit cube $[0, 1]^d$ with d an integer and m points in it distributed randomly. How big should be the length l of the smallest hyper-cube to contain at least 1 point? We can easily calculate it as

$$l^d \approx \frac{1}{m} \rightarrow l \approx \left(\frac{1}{m}\right)^{1/d} \quad (23)$$

We can easily calculate this value of l for various values of d . Let us suppose that we consider $m = 1000$ and summarize the results in Table 3.

d	l
2	0.003
10	0.50
100	0.93
1000	0.99

Table 3: Length l of the smallest hyper-cube to contain at least 1 point from a population of randomly distributed m points.

Furthermore, as you can see the data becomes so sparse in high dimensions that you need to consider the entire hyper cube to capture one single observation. When the data becomes so sparse the number of observations you will need to train an algorithm properly becomes much bigger than the size of existing datasets. We could look at this differently. Let us consider now a small hypercube of side $l = 1/10$. How many observations we will find on average in this small portion of the hypercube? This is easy to calculate and is given by

$$\frac{m}{10^d} \quad (24)$$

You can see that this number is very small for high values of d . For example, if we consider $d = 100$ is easy to see that we would need more observations than atoms in the universe¹⁷ to find at least one observation in that small portion of the hypercube.

¹⁷<https://www.universetoday.com/36302/atoms-in-the-universe/>

Note Doing dimensionality reduction is a very viable method for reducing dramatically running time while incurring in an only small drop in accuracy. In high dimensionality datasets this becomes fundamental due to the curse of dimensionality.

3.3 Anomaly Detection Egor Zhestov (@yk4r2)

Autoencoders are often used to perform anomaly detection on the most different datasets. The best way to understand how anomaly detection works with autoencoders is to look at it with a practical example. Let us consider an autoencoder with only three layers with 784 neurons in the first, 64 in the latent feature generation layer, and again 784 neurons in the output layers. We will train it with the MNIST dataset and in particular with the 60000 training portion of it as we have done in the previous sections of the article. Now let us consider the Fashion MNIST dataset. Let us choose an image of a shoe (see Figure 9) from this dataset and add it to the testing portion

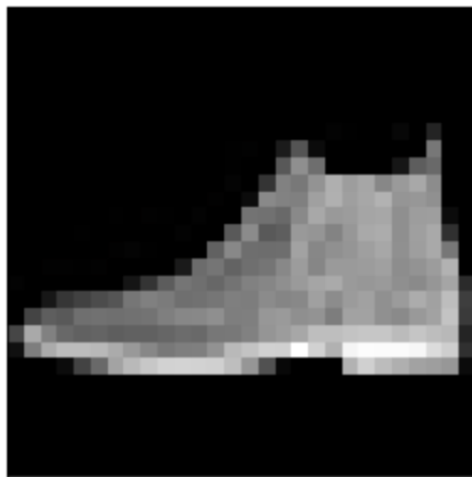


Figure 9: one random image from the Zalando MNIST dataset.

of the MNIST dataset. The original testing portion of MNIST has 10000 images. With the shoe we will have a 10001 images dataset. How can we use an autoencoder to find the shoe automatically in those 10001 images? Note that the shoe is an "outlier", an "anomaly" since it is an entirely different

image class than hand-written digits. To do that we will take the autoencoder we trained with the 60000 MNIST images and with it we will calculate the reconstruction error for the 10001 test images.

The main idea is that since the autoencoder has only seen hand-written digits images, it will not be able to reconstruct the shoe image. Therefore we expect this image to have the biggest reconstruction error. We can check if that is the case by taking the top 2 reconstruction errors. For this example, we have used the MSE for the reconstruction error. You can check the code of this example at <https://ad1.toelt.ai>. The shoe has the highest reconstruction error: 0.062. The autoencoder is not able to reconstruct the image as it can be seen from Figure 10.

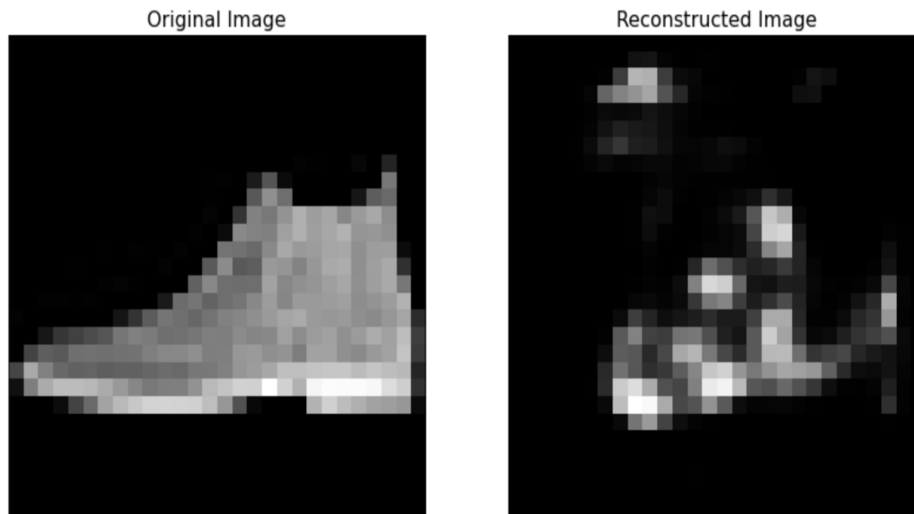


Figure 10: The shoe and the autoencoder's reconstruction trained on the 60000 hand-written images of the MNIST dataset. **This image has the biggest RE in the entire 10001 test dataset** we built with a value of 0.062.

The second biggest RE is slightly less than one third of that of the shoe: 0.022, indicating that the autoencoder is doing quite a good job in understanding how to reconstruct hand-written digits. You can see the image with the second biggest RE in Figure 11. This image could also be classified as an outlier, as is not completely clear if it is a 4 or an incomplete 9. The readers with most experience may have noticed that we trained our autoencoders on a dataset without any outliers and applied it to a second dataset with outliers. This is not always possible as very often the outliers are not known

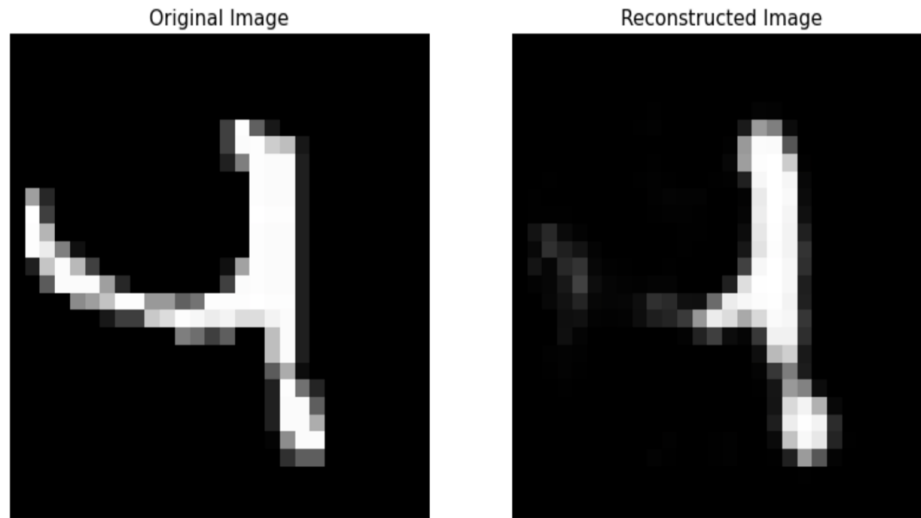


Figure 11: the image with the second biggest RE in the 10001 test dataset: 0.022.

and are lost in a big dataset. In general, one wants to find outliers in a single big dataset without any information on how many there are or how they look like. Generally speaking, anomaly detection can be done following the main steps below.

- One train an autoencoder on the entire dataset (or if possible, on a portion of the dataset known **not** to have any outlier).
- For each observation (or input) of the portion of the dataset known to have the wanted outliers one calculates the RE.
- One sorts the observations by the RE.
- One classifies the observations with the highest RE as outliers. Note that how many observations are outliers will depend on the problem at hand and require an analysis of the results and usually lot of knowledge of the data and the problem.

Note that if one train the autoencoder on the entire dataset at disposal, there is an essential assumption: the outliers are a negligible part of the dataset and their presence will not influence (or will influence in an insignificant way) how the autoencoder learns to reconstruct the observations. This is one of

the reasons why regularization is so essential. If the autoencoders would learn the identity function, anomaly detection could not be done. A classic example of anomaly detection is finding fraudulent credit card transactions (the outliers). This case usually presents ca. 0.1% fraudulent transactions and therefore this would be a case that would allow us to train the autoencoder on the entire dataset. Another is fault detection in industrial environment.

Note If one train the autoencoder on the entire dataset at disposal, there is an essential assumption: the outliers are a negligible part of the dataset and their presence will not influence (or will influence in an insignificant way) how the autoencoder learns to reconstruct the observations.

3.3.1 Model Stability – a short note

Note that doing anomaly detection as described in the previous section seems easy, but those methods are prone to overfitting and give often inconsistent results. This means that training an autoencoder with a different architecture may well give different REs and therefore other outliers. There are several ways of solving this problem, but one of the simplest ways of dealing with instability of results is to train different models and then take the average of the REs. Another often used technique involves taking the maximum of the REs evaluated from several models.

Note Anomaly detection done with autoencoders is prone to problems as overfitting and unstable results. It is essential to be aware of these problems and check the results coming from different models to interpret the results correctly.

Note that this section serves to give you some pointers and is not meant to be an exhaustive overview on how to solve this problem. Like autoencoders ensembles¹⁸, more advanced techniques are also used to deal with problems of instable results coming, for example, from small datasets.

¹⁸See for example <https://saketsathe.net/downloads/autoencode.pdf>.

3.4 Denoising autoencoders Go to 'AutoEncoders.pdf', page 4

Denoising autoencoders¹⁹ are developed to auto-correct errors (noise) in the input observations. As an example, imagine the hand-written digits we considered before where we added some noise (for example Gaussian noise) in the form of changing randomly the gray values of the pixels. In this case the autoencoders should learn to reconstruct the image without the added noise. As a concrete example, consider the MNIST dataset. We can add to each pixel a random value generated by a normal distribution scaled by a factor (you can check the code at <https://adl.toelt.ai> in article 25). We can train an autoencoder using as input the noisy images, and as output the original images. The model should learn to remove the noise, since it is random in nature and have no relationship with the images. In Figure 12 you can see the results. In the left column you see the noisy images, in the middle the original ones and on the right the de-noised images. It is quite impressive how well it works. Figure 12 has been generated by training a FFA autoencoder with 3 layers and 32 neurons in the middle layer. Go to 'AutoEncoders.pdf', page 11

4 Beyond FFA – autoencoders with convolutional layers

In this article we have described autoencoders with a feed-forward architecture. **There is no rule and autoencoders with convolutional layers works as well, and often (especially when dealing with images) are much more efficient.** For example, in Figure 13 you can see a comparison of the results of a FFA (with architecture (784,32,784)) and of a Convolutional Autoencoder (CA) (with architecture ((28x28), (26x26x64), (24x24,32), (26x26x64), (28x28)); keep in mind the layers are convolutions, so the first two numbers indicate the tensor dimensions and the third the number of kernels, that in this example had a size of 3x3). The two autoencoders have been trained with the same parameters (epochs, mini-batch size, etc.). You can see how a CA gives better results than a FFA, since we are dealing with images. To be fair, note that the feature generating layer is only marginally smaller than

¹⁹Vincent, P., Larochelle, H. Bengio, Y. Manzagol, P.A.: Extracting and composing robust features with denoising autoencoders. In: Proceedings of the 25th International Conference on Machine Learning, ICML '08, pp. 1096-1103. ACM, New York, NY USA (2008)

the input layer in this example. The purpose of this example is only to show you how also convolutional autoencoders are a viable solution that works very well in many practical applications.

Another important aspect is that the feature generating layer could be a convolutional layer but could also be a dense one. There is not fix rule and testing is required to find the best architecture for your problem and how you want to model your latent features: as a tensor (multi-dimensional array) or as a one-dimensional array of real numbers.

5 Code Examples

On <https://adl.toelt.ai> you will find examples of autoencoders, anomaly detection with autoencoders and denoising with autoencoders as described in this article.

6 Further Readings

- Deep Learning Tutorial from Stanford University
<http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/>
- Building autoencoders in Keras
<https://blog.keras.io/building-autoencoders-in-keras.html>
- Introduction to autoencoders in TensorFlow
<https://www.tensorflow.org/tutorials/generative/autoencoder>
- Bank, D., Koenigstein, N., and Giryas, R., "Autoencoders", arXiv e-prints, 2020,
<https://arxiv.org/abs/2003.05991>
- R. Grosse, University of Toronto, Lecture on autoencoders
http://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec20.pdf

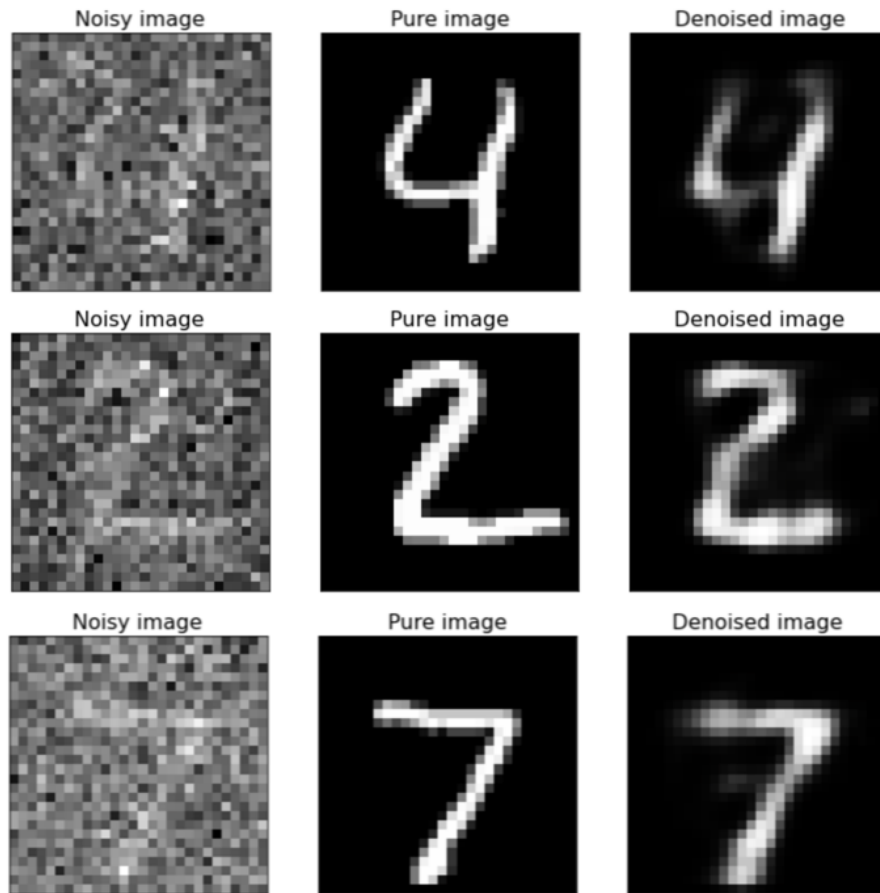


Figure 12: Results of a denoising FFA autoencoder with 3 layers and 32 neurons in the middle layer. The noise has been generated adding a real number between 0 and 1 taken from a normal distribution. For details see the code at <https://adl.toelt.ai>.

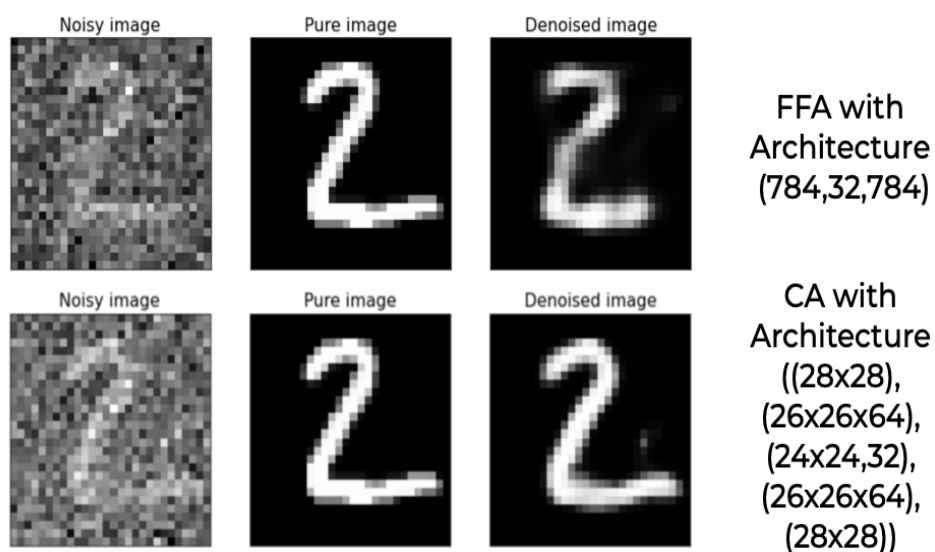


Figure 13: Comparison of the results of a FAA (with architecture (784,32,784)) and of a Convolutional Autoencoder (CA) (with architecture ((28x28), (26x26x64), (24x24,32), (26x26x64), (28x28)); keep in mind the layers are convolutions, so the first two numbers indicate the tensor dimensions and the third the number of kernels, with kernel size 3x3). The two autoencoders have been trained with the same parameters. You can check the code at <https://adl.toelt.ai>.